UNIT 3

Q1) What is context free grammar?
Let G = (V, T, P, S) be the context-free grammar. If A → β is a production of P and α and β are any strings from non-terminals or terminals i.e., in (VUT)*, then α A β ⇒ α B β.

Suppose a1, a2, a3, ..., αm are strings in (V U T)*, m ≥ 1.

- a1 ⇒ a2
- a2 ⇒ a3
- ...
- $\mathbf{a_{m-1}} \Rightarrow \mathbf{a_m}$

Then we can say that $a_1 \overset{*}{\underset{G}{\Rightarrow}} a_m$.

Q2) Write a note on Chomsky hierarchy.

Language class	Language	Grammar	Machine	One example
Type 3	Regular	Regular grammar	FSM i.e. NFA or DFA	a^*b^*
Type 2	Context free	Context free grammar	PDA	$a^n b^n$
Type 1	Decidable languages	Context sensitive grammar	Linear bounded automata	$a^n b^n c^n$
Type 0	Computable languages	Unrestricted grammar	Turing machine	n!

Type 3 - Regular languages
Regular languages are those languages which can be described using regular expressions. These languages can be modeled by NFA or DFA.
Type 2 - Context-free languages
The context-free languages are the languages which can be represented by Context Free Grammar (CFG). The production rule is of the form: **A -> α**
where A is any single non-terminal and α is any combination of terminals and non-terminals.

An NFA or DFA cannot recognize strings of this language because these automata do not have a "stack" to memorize. Instead of it, the Push Down Automata can be used to represent these languages.

Type 1 - Context-sensitive languages
The context-sensitive grammars are used to represent context-sensitive languages. The context-sensitive grammar follows the following rules:
1. The number of symbols on the left hand side of their production rules must not exceed the number of symbols on the right hand side.
2. The rule of the form A → ε is not allowed unless A is a start symbol. It does not occur on the right hand side of any rule.
The automaton which recognizes context-sensitive languages is called linear bounded automaton. While deriving using context-sensitive grammar the sentential form must always increase in length every time a production rule is applied. Thus the size of a sentential form is bounded by a length of the sentence we are deriving.

Type 0 - Unrestricted languages : There is no restriction on the grammar rules of these types of languages. These languages can be effectively modeled by Turing machines.

Q3) **Ambiguous Grammar**

Definition: A grammar is said to be **ambiguous** , if there exists more than one parse trees for a given grammar, that means there could be more than one leftmost or rightmost derivation possible .This means the grammar does not uniquely define the structure of the string.

Explanation: Ambiguity arises when a string can be derived in multiple ways, leading to different interpretations. Ambiguity is problematic in applications like programming languages, where a unique interpretation of a program is essential.

The grammar can be derived in either leftmost derivation or rightmost derivation. One can draw a derivation tree called as **Parse tree** or **syntax tree** based on these deriavtions. The parse tree has to be unique even through the derivation is leftmost or rightmost.

Consider the grammar:

1. $E \to E + E$
2. $E \to E * E$
3. $E \to a$

This grammar generates arithmetic expressions using $+$, $*$, and the operand a.

String: a + a $*$ a

We can derive the string a + a $*$ a in two ways:

1. **Parse Tree 1** (Interpret $+$ first):

mathematica

```
      E
    / | \
   E  +  E
  /     / | \
 a     E  *  E
         a     a
```

2. **Parse Tree 2** (Interpret $*$ first):

mathematica

```
      E
    / | \
   E  *  E
  /     / | \
 E     a   +  E
 a         a
```

Q4) **Left Linear and Right Linear Regular Grammars**

Definition: A **regular grammar** is classified into two types based on the position of the non-terminal symbol in the production rules:

1. Left Linear Grammar:

In a left-linear grammar, the non-terminal appears on the left side of the string in the production rules.

Form: A→B or A→a, where A,B are non-terminals, and a is a terminal.

Example:

S→aS

S→b

1.Right Linear Grammar:

In a right-linear grammar, the non-terminal appears on the right side of the string in the production rules.

Form: A→aB or A→a, where A,B are non-terminals, and a is a terminal.

Example:

S→Sa

S→b

Conversion of Left Linear Grammar to Right Linear Grammar

1.Construct transition graph for given grammar.

2. Interchange initial and final states.

3. Reverse the directions of all the edges in the graph.

4. Write right linear grammar for newly formed transition graph.

Example Left Linear Grammar:

1. S→aA

2. A→b
3. A→c

Step 1: Construct the Transition Graph

- States: S, A
- Transitions:
  - $S \xrightarrow{a} A$ (from $S \to aA$)
  - $A \xrightarrow{b} A$ (from $A \to bA$)
  - $A \xrightarrow{c} \text{Final}$ (from $A \to c$)

Transition Graph:

```
S --a--> A
A --b--> A
A --c--> Final
```

Step 3: Reverse the Directions of All Edges

Reverse all edges from the original graph:

1. $A \xrightarrow{a} S$ (reverse $S \xrightarrow{a} A$).
2. $A \xrightarrow{b} A$ (self-loop remains the same).
3. $\text{Final} \xrightarrow{c} A$ (reverse $A \xrightarrow{c} \text{Final}$).

Reversed Transition Graph:

```
Final --c--> A
A --b--> A
A --a--> S
```

Step 2: Interchange Initial and Final States

- **Original Initial State:** S
- **Original Final State(s):** A (as it transitions to c).

After interchanging:

- New Initial State: $Final$.
- New Final State: S.

Step 4: Write the Right Linear Grammar

From the reversed transition graph, construct the Right Linear Grammar:

1. $Final \to cA$
2. $A \to bA$
3. $A \to aS$
4. $S \to \epsilon$ (as S is the new final state).

Conversion of Right Linear Grammar to Left Linear Grammar

1. Convert the given right linear grammar to equivalent FA.
2. From the FA, interchange the initial state and the final state.
3. Reverse the arrows on all edges.
4. Construct the regular grammar from this FA.

Given Right Linear Grammar:

1. S→aA
2. A→b
3. A→c

Step 1: Convert the RLG to an Equivalent Finite Automaton (FA)

States:

S: Start state
A: Intermediate state
F: Final state (implicit for terminal productions).

Transitions:

1. $S \xrightarrow{a} A$ (from $S \to aA$).
2. $A \xrightarrow{b} A$ (from $A \to bA$).
3. $A \xrightarrow{c} F$ (from $A \to c$).

Finite Automaton (FA):

```
S --a--> A
A --b--> A
A --c--> F
```

Step 2: Interchange Initial and Final States

- **Original Initial State:** S
- **Original Final State(s):** F.

After interchanging:

- New Initial State: F.
- New Final State: S.

Step 3: Reverse the Arrows on All Edges

Reverse the direction of all transitions:

1. $A \xrightarrow{a} S$ (reverse $S \xrightarrow{a} A$).
2. $A \xrightarrow{b} A$ (self-loop remains the same).
3. $F \xrightarrow{c} A$ (reverse $A \xrightarrow{c} F$).

Reversed Transition Graph:

```
F --c--> A
A --b--> A
A --a--> S
```

Step 4: Construct the Left Linear Grammar

From the reversed FA, construct the equivalent **Left Linear Grammar (LLG):**

1. $S \to aA$ (from $A \xrightarrow{a} S$).
2. $A \to bA$ (from $A \xrightarrow{b} A$).
3. $A \to cF$ (from $F \xrightarrow{c} A$).

Final Left Linear Grammar:

1. $S \to aA$
2. $A \to bA$
3. $A \to cF$

Q5) **Regular Grammar**

Definition: A **Regular Grammar** is a formal grammar that generates a **regular language**. It consists of a finite set of production rules that describe how strings in a language can be formed using terminals (symbols from the language) and non-terminals (intermediate variables).

G = (V, T, P, S) where

V is a set of symbols called non-terminals which are used to define the rules.

T is a set of symbols called terminals.

P is a set of production rules.

S is a start symbol which ∈ V.

Characteristics of Regular Grammar:

Each production rule has one of the following form:

A→aB or A→a (Right Linear Grammar)

A→Ba or A→a (Left Linear Grammar) Here, A and B are non-terminals, and a is a terminal.

A regular grammar cannot generate languages that require memory or recursion, such as $\mathbf{a^n b^n}$

It can be implemented using Finite Automata.

Types of Regular Grammar

Right Linear Grammar (RLG):

The non-terminal appears at the rightmost position in the production.

Form: A→aB or A→a

Commonly used in programming languages and lexical analysis.

Example:

S→aA

A→bB

B→c

2. **Left Linear Grammar (LLG):**

The non-terminal appears at the leftmost position in the production.

Form: A→Ba or A→aA

Example:

S→Aa

A→Bb

B→c

Q6) Prove that CFL's are closed under union, concatenation, Kleene's closure.

1. Union of CFLs

Statement:

If L_1 and L_2 are context-free languages, then their union $L = L_1 \cup L_2$ is also context-free.

Proof:

- Let $G_1 = (N_1, T, P_1, S_1)$ and $G_2 = (N_2, T, P_2, S_2)$ be the context-free grammars for L_1 and L_2, respectively.
- Construct a new grammar $G = (N, T, P, S)$, where:
  - $N = N_1 \cup N_2 \cup \{S\}$ (combine non-terminals from G_1, G_2, and add a new start symbol S).
  - $T = T$ (the terminal symbols are the same for both grammars).
  - $P = P_1 \cup P_2 \cup \{S \to S_1 \mid S_2\}$ (include the production rules of G_1 and G_2, and add a rule for S to choose either S_1 or S_2).
- The new grammar G generates $L_1 \cup L_2$.

Conclusion:

CFLs are closed under union.

2. Concatenation of CFLs

Statement:

If L_1 and L_2 are context-free languages, then their concatenation $L = L_1 \cdot L_2$ is also context-free.

Proof:

- Let $G_1 = (N_1, T, P_1, S_1)$ and $G_2 = (N_2, T, P_2, S_2)$ be the context-free grammars for L_1 and L_2, respectively.
- Construct a new grammar $G = (N, T, P, S)$, where:
  - $N = N_1 \cup N_2 \cup \{S\}$ (combine non-terminals from G_1, G_2, and add a new start symbol S).
  - $T = T$ (the terminal symbols are the same for both grammars).
  - $P = P_1 \cup P_2 \cup \{S \to S_1 S_2\}$ (include the production rules of G_1 and G_2, and add a rule for S to produce $S_1 S_2$).
- The new grammar G generates strings in $L_1 \cdot L_2$, as it first derives a string from L_1 (using S_1) and then a string from L_2 (using S_2).

Conclusion:

CFLs are closed under concatenation.

3. Kleene's Closure of CFLs

Statement:

If L_1 is a context-free language, then its Kleene closure $L = L_1^*$ is also context-free.

Proof:

- Let $G_1 = (N_1, T, P_1, S_1)$ be the context-free grammar for L_1.
- Construct a new grammar $G = (N, T, P, S)$, where:
  - $N = N_1 \cup \{S\}$ (combine non-terminals from G_1 and add a new start symbol S).
  - $T = T$ (the terminal symbols remain the same).
  - $P = P_1 \cup \{S \to S_1 S \mid \epsilon\}$ (include the production rules of G_1, and add a rule for S to produce either $S_1 S$ or ϵ).
- The new grammar G generates strings in L_1^*, as it repeatedly derives strings from L_1 (using S_1) or stops with ϵ.

Conclusion:

CFLs are closed under Kleene's closure.

4. CFLs are Not Closed Under Intersection

Proof: To prove CFLs are **not closed under intersection**, we provide a counterexample.

Let $L_1 = \{a^n b^n c^m \mid n, m \geq 0\}$ and $L_2 = \{a^m b^n c^n \mid n, m \geq 0\}$, both of which are context-free languages.

The intersection $L_1 \cap L_2$ is the set of strings where the number of a's, b's, and c's are all equal, i.e., $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$.

The language $\{a^n b^n c^n \mid n \geq 0\}$ is **not a context-free language**, as it cannot be generated by a context-free grammar due to the requirement of matching three different symbols simultaneously. It is, in fact, a context-sensitive language.

Since $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$ is not a context-free language, this shows that CFLs are **not closed under intersection**.

Q7)**Pumping Lemma for Context-Free Languages (CFLs)**

The Pumping Lemma for Context-Free Languages (CFLs) is a property used to prove that certain languages are not context-free. It is a theoretical tool that applies to all context-free languages and helps us understand the structure of CFLs. This lemma is used in a similar way to the pumping lemma for regular languages, but it is more complex due to the nature of context-free languages.

Statement of the Pumping Lemma for CFLs

Let L be a context-free language. Then, there exists a constant p (called the **pumping length**) such that any string $w \in L$ of length at least p can be divided into five parts, $w = uvxyz$, satisfying the following conditions:

1. **Length condition**: $|vxy| \leq p$ (the length of the middle part, vxy, is at most p).
2. **Non-empty condition**: $|vy| \geq 1$ (the substrings v and y are non-empty, i.e., they must contain at least one symbol).
3. **Pumping condition**: For all $i \geq 0$, the string $uv^i xy^i z$ is in L. This means that repeating (or removing) the substrings v and y any number of times (i.e., i times) must still produce strings that belong to L.

Interpretation of the Lemma: The pumping lemma tells us that for a sufficiently long string www in a context-free language, there is a way to "pump" or repeat parts of the string (specifically the substrings vvv and yyy) while still ensuring that the resulting string remains in the language. The pumping lemma is used to show that certain languages cannot be context-free by finding strings in the language that cannot be pumped according to the conditions of the lemma.